



THE COPPERBELT UNIVERSITY
SCHOOL OF INFORMATION AND COMMUNICATION
TECHNOLOGY
COMPUTER SCIENCE DEPARTMENT

System Design

A Search-Based Intelligent Agent for the Traditional Board Game Nsolo

Student Name: Nanetu E Mushongoma

Student ID: 22102890

Programme: Bachelor Of Science in Computer Science

Supervisor: Mr. Phillimon Mumba

Date: 8th March, 2026

CHAPTER 4: SYSTEM DESIGN.....	2
4.1 Introduction.....	2
4.2 Formalisation of Nsolo Game Rules.....	3
4.2.1 Overview.....	3
4.2.2 Board Structure.....	3
4.2.3 Initial Configuration.....	4
4.2.4 Sowing Direction.....	5
4.2.5 Move Selection and Legality.....	6
4.2.6 The Sowing Action.....	7
4.2.7 Relay Sowing.....	7
4.2.8 Capture Rule.....	8
4.2.9 Game Termination and Win Condition.....	10
4.2.10 Summary of Formal Specification.....	10
4.3 Analysis of the System.....	11
4.4 Context Model of the System.....	12
4.5 Design Methods.....	14
4.5.1 Architectural Design.....	14
4.5.2 Detailed Object-Oriented Design.....	16
4.5.3 Key Interaction Sequences.....	18
4.5.4 State Machine Design.....	20
4.6 Physical Design.....	21
4.6.1 User Interface Design.....	21
4.6.2 Data Model.....	22
4.6.3 Data Flow Through the System.....	22
4.7 Conclusion.....	23
REFERENCES.....	25

LIST OF FIGURES

Figure 4.1: Nsolo 4×12 board layout showing row assignments for each player, inner and outer row designations, and column indexing from 0 to 11	3
Figure 4.2: Diagram illustrating a relay sowing sequence across five sowing steps. The initial hole (r=1, c=5) is selected; stones are sown counterclockwise; the last stone lands in a non-empty hole, triggering relay; relay continues until termination at an empty hole at (r=1, c=0).....	7
Figure 4.3: Capture scenario diagram showing Player 1's sowing chain terminating at inner row hole (1, 7), which was empty. Arrows indicate the capture of stones from Player 2's inner row hole (2, 7) and outer row hole (3, 7).....	9
Figure 4.4: Context Diagram for the Nsolo Game System showing external entities (Human Player, Android OS, Unity Engine) and the primary data flows into and out of the system boundary.....	13
Figure 4.5: Architectural Diagram of the Layered Architecture of the Nsolo Game System showing the Presentation Layer (Unity UI), Application Logic Layer (Game Controller, Game Engine, AI Agent), and the Android Platform.....	14
Figure 4.6: UML Class Diagram showing GameBoard, GameEngine, SowingPath, AI Agent, EvaluationFunction, GameController, and UIManager with their attributes, methods, and associations.....	17
Figure 4.7: UML Sequence Diagram of Human Player Turn showing interactions between UIManager, GameController, GameEngine, and GameBoard.....	18
Figure 4.8: UML Sequence Diagram of AI Player Turn showing interactions between GameController, AI Agent, EvaluationFunction, TranspositionTable, and GameEngine.....	19
Figure 4.9: State Machine Diagram for the Nsolo Game Controller showing the six game states and the events that trigger transitions between them.....	20
Figure 4.10: User Interface Wireframe showing the 4×12 board layout, score counters, turn indicator, and menu icon placement on a standard Android portrait screen.....	21
Figure 4.11: Data Flow Diagram (Level 1) for the Nsolo Game System showing the Input Handler, Game Engine, AI Decision Engine, and Score Tracker processes with their data stores and flows.....	22

CHAPTER 4: SYSTEM DESIGN

4.1 Introduction

This chapter presents the system design for the Nsolo intelligent agent application. Building directly on the requirements specification and system analysis in Chapter 3, this chapter addresses the structural and behavioural realisation of the system: how the identified components are organised, how they interact, and how the design ensures correctness, performance, and maintainability within the constraints of mobile deployment.

The design is structured according to object-oriented design principles and is informed by the separation of concerns established in the methodology. The core game engine and AI components are designed as a Unity-independent C# library to enable correctness verification prior to integration, while the presentation layer is implemented within the Unity game engine. Additionally, all three-dimensional visual assets, including the board and stones, are modelled and optimised using Blender before being imported into Unity.

The chapter proceeds as follows. Section 4.2 presents the formalisation of Nsolo game rules as a computational specification. Section 4.3 analyses the system in terms of functional responsibilities and design constraints. Section 4.4 defines the system context and external interactions. Section 4.5 presents the architectural and detailed object-oriented design, including subsystem decomposition and interaction sequences. Section 4.6 describes the physical design, covering the user interface and data model. The chapter concludes in Section 4.7 with a synthesis of how the design satisfies the requirements established in Chapter 3.

4.2 Formalisation of Nsolo Game Rules

4.2.1 Overview

A computational implementation of any traditional board game must rest on a precise, unambiguous specification of its rules. Without this, neither the correctness of the implementation nor the validity of the AI agent's decisions can be established, since the minimax search evaluates and compares thousands of potential board positions against the same rule logic. This section presents the formalised rules of Nsolo derived from primary data collection with experienced players in Kitwe, Copperbelt Province, cross-referenced

against Sanderson (1913), Townshend (1975), and Russ (1984). The formalisation is presented as a structured computational specification: each rule is stated in natural language, followed by a formal definition using mathematical or pseudocode notation, and a note on its implementation significance.

The rules presented below represent a consistent and commonly observed version of Nsolo gameplay, annotated where regional variants were observed during data collection.

4.2.2 Board Structure

The Nsolo board consists of four rows and twelve columns, yielding forty-eight holes in total. Each hole is identified by a two-dimensional index (r, c) where $r \in \{0, 1, 2, 3\}$ denotes the row and $c \in \{0, 1, \dots, 11\}$ denotes the column. The board is divided vertically into two sides: Player 1 controls rows 0 and 1, and Player 2 controls rows 2 and 3.

For each player, the row closer to the opponent is designated the inner row, and the row farther from the opponent is designated the outer row. Under this assignment:

- Player 1 inner row: row 1 (adjacent to Player 2's side).
- Player 1 outer row: row 0.
- Player 2 inner row: row 2 (adjacent to Player 1's side).
- Player 2 outer row: row 3.

The formal board state is defined as a two-dimensional integer array B of dimension 4×12 , where $B[r][c] \geq 0$ for all (r, c) denotes the number of stones currently occupying hole (r, c) . The state of the board at any point in the game is fully described by B .

Figure 4.1 illustrates the board layout with row and player assignments.

Figure 4.1: Nsolo 4×12 Board Layout



Figure 4.1: Nsolo 4×12 board layout showing row assignments for each player, inner and outer row designations, and column indexing from 0 to 11.

4.2.3 Initial Configuration

At the start of the game, each hole is initialised with two stones, with the exception of one designated empty hole per player in their respective inner row. Based on the most consistently observed practice among interviewed players in Kitwe, the empty hole is the rightmost hole of each player's inner row. The initial configuration is therefore:

```

 $B[r][c] = 2$       for all  $(r, c) \in \{0, 1\} \times \{0, \dots, 11\}$  except  $(1, 11)$ 
 $B[r][c] = 2$       for all  $(r, c) \in \{2, 3\} \times \{0, \dots, 11\}$  except  $(2, 0)$ 
 $B[1][11] = 0$ 
 $B[2][11] = 0$ 

```

The initial total stone count per player is therefore $2 \times 12 \times 2 - 2 = 46$ stones each, giving a total of 92 stones on the board at game start. This total is conserved until captures begin reducing it.

Note: Some communities were observed to place the initial empty hole at the leftmost position rather than the rightmost. This implementation uses the rightmost convention as the canonical form, with a configuration constant that may be adjusted for regional variation.

4.2.4 Sowing Direction

Nsolo uses a counterclockwise sowing direction, which is the standard for the majority of Mancala variants played in Central and Southern Africa (Russ, 1984). To implement counterclockwise sowing correctly, the traversal order around the board must be defined precisely.

The board is traversed as a closed loop of 48 holes in the following order:

For Player 1, the traversal path is defined as follows:

- Along the outer row (row 0) from column 0 to column 11 (left to right)
- Along the inner row (row 1) from column 11 to column 0 (right to left)

For Player 2, the traversal path is defined as:

- Along the inner row (row 2) from column 0 to column 11 (left to right)
- Along the outer row (row 3) from column 11 to column 0 (right to left)

Each traversal forms a closed loop of 24 holes within the player's territory. The loop wraps within the player's rows only and does not extend into the opponent's rows.

This traversal is encoded as a player-specific ordered sequence of (row, column) pairs, denoted `PATH_p`. Given a starting hole at index `i` in `PATH_p`, sowing proceeds along `PATH_p[i+1]`, `PATH_p[i+2]`, and so on, wrapping modulo 24. The starting hole itself is excluded from sowing, meaning no stone is placed back into the hole from which stones were picked up at the beginning of the current relay segment.

```
PATH = [
    (0,0), (0,1), ..., (0,11),    // Player 1 outer row, left to right
    (1,11), (1,10), ..., (1,0),   // Player 1 inner row, right to left
    (2,0), (2,1), ..., (2,11),    // Player 2 inner row, left to right
    (3,11), (3,10), ..., (3,0)    // Player 2 outer row, right to left
]

function sow(B, start_index, stones):
    i = start_index
    while stones > 0:
        i = (i + 1) mod 48
        B[PATH[i].row][PATH[i].col] += 1
        stones -= 1
    return i    // index of last hole to receive a stone
```

The sowing direction is an implementation invariant: any deviation from this traversal constitutes a rule violation. The `PATH` array is defined once at initialisation and referenced throughout the game engine.

4.2.5 Move Selection and Legality

A player's turn begins by selecting a hole from which to sow. The selected hole must satisfy two conditions:

- It must belong to the current player's side of the board (rows 0-1 for Player 1, rows 2-3 for Player 2).
- It must contain at least one stone ($B[r][c] \geq 1$).

Single-stone holes are legal selections without restriction. Players are not prohibited from selecting such holes, though strategically it is generally unfavourable to do so unless no multi-stone hole exists. The legal move set for player p at board state B is:

```
LEGAL_MOVES(B, p) = {  
  (r, c) | r ∈ player_rows(p) AND B[r][c] >= 1  
}  
  
player_rows(1) = {0, 1}  
player_rows(2) = {2, 3}
```

If $\text{LEGAL_MOVES}(B, p)$ is empty, the game ends immediately and the opposing player wins, as defined in Section 4.2.9.

4.2.6 The Sowing Action

When a player selects hole (r, c) , the following action is executed:

1. All stones are removed from hole (r, c) : let $n = B[r][c]$, then $B[r][c] \leftarrow 0$.
2. The index of (r, c) in PATH is located: let $i = \text{PATH_INDEX}(r, c)$.
3. The sow function is executed: $\text{last_index} \leftarrow \text{sow}(B, i, n)$.
4. Relay sowing and capture checks are evaluated as defined in Sections 4.2.7 and 4.2.8.

No stone is deposited back into the hole (r, c) during the initial sowing phase of this action. However, if the sowing path wraps around the board and reaches (r, c) again in a subsequent relay, it is treated as any other hole on the path and receives a stone normally.

4.2.7 Relay Sowing

Relay sowing is the defining mechanic of Nsolo and distinguishes it from simpler Mancala variants. After the sow function returns last_index :

- Let $(r_l, c_l) = \text{PATH}[\text{last_index}]$ be the last hole to receive a stone.
- If $B[r_l][c_l] > 1$ (the hole contained stones before the final stone was placed, and now contains more than one stone), the relay condition is met.
- All stones are immediately picked up from (r_l, c_l) : let $n = B[r_l][c_l]$, then $B[r_l][c_l] \leftarrow 0$.
- Sowing resumes from last_index : $\text{last_index} \leftarrow \text{sow}(B, \text{last_index}, n)$.
- This process repeats until the relay condition is no longer met.

The relay terminates when the last stone lands in a hole that was empty before receiving that stone, i.e., $B[r_l][c_l] = 1$ after sowing (meaning it was 0 before). At this point, the capture check is evaluated.

```
function execute_turn(B, r, c):
    n = B[r][c]
    B[r][c] = 0
    i = PATH_INDEX(r, c)
    last = sow(B, i, n)

    while B[PATH[last].row][PATH[last].col] > 1:
        n = B[PATH[last].row][PATH[last].col]
        B[PATH[last].row][PATH[last].col] = 0
        last = sow(B, last, n)

    evaluate_capture(B, PATH[last], current_player)
```

The relay mechanic substantially increases the strategic depth of Nsolo relative to two-row Mancala variants, as a single initial selection can produce chains of redistribution affecting large portions of the board. It is also the primary source of the game's computational complexity: predicting the outcome of a relay chain requires simulating multiple sequential sowing operations, meaning that the effective branching factor in the search tree is not simply the number of initial legal moves but depends on the board state in a non-trivial manner.

Figure 4.3: Relay Sowing Sequence - Player 1, Starting at (r=1, c=5)

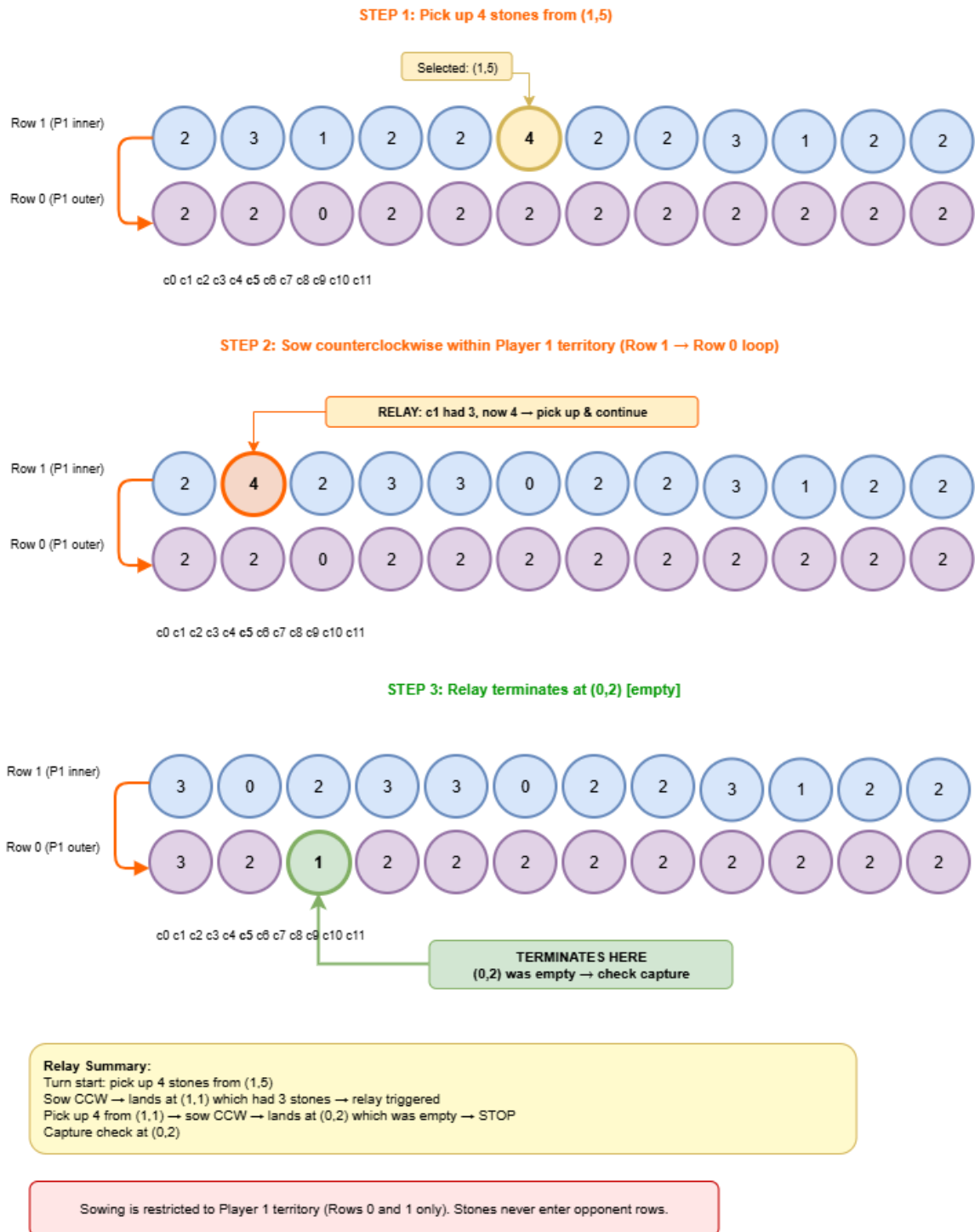


Figure 4.2: Diagram illustrating a relay sowing sequence across five sowing steps. The initial hole (r=1, c=5) is selected; stones are sown counterclockwise; the last stone lands in a non-empty hole, triggering relay; relay continues until termination at an empty hole at (r=1, c=0).

4.2.8 Capture Rule

A capture is evaluated immediately after the relay terminates. Let (r_f, c_f) be the final hole at which the last stone was placed. A capture occurs if and only if the following two conditions are simultaneously satisfied:

- Condition A - Inner row landing: (r_f, c_f) is in the current player's inner row. For Player 1: $r_f = 1$. For Player 2: $r_f = 2$.
- Condition B - Empty landing: the hole (r_f, c_f) was empty before the final stone was placed, i.e., $B[r_f][c_f] = 1$ after sowing (it received exactly one stone into an empty hole).

If both conditions are met, the following stones are captured:

1. Primary capture: all stones in the opponent's inner row hole in the same column c_f are taken. For Player 1 capturing: stones at $(2, c_f)$. For Player 2 capturing: stones at $(1, c_f)$.
2. Extended capture (outer row): if the opponent's outer row hole in column c_f contains stones, those are also captured. For Player 1 capturing: stones at $(3, c_f)$ if $B[3][c_f] > 0$. For Player 2 capturing: stones at $(0, c_f)$ if $B[0][c_f] > 0$.

All captured stones are removed from the board and added to the current player's captured stone total. The captured stone count is maintained as a separate integer per player and is not represented on the board.

```
function evaluate_capture(B, final_hole, player):
    (r_f, c_f) = final_hole
    inner_row = {1: 1, 2: 2}[player]

    if r_f != inner_row:
        return // No capture: did not land in inner row

    if B[r_f][c_f] != 1:
        return // No capture: hole was not empty before last stone

    // Capture opponent's inner row hole
    opp_inner = {1: 2, 2: 1}[player]
    opp_outer = {1: 3, 2: 0}[player]

    captured += B[opp_inner][c_f]
    B[opp_inner][c_f] = 0

    // Extended capture: opponent's outer row
    if B[opp_outer][c_f] > 0:
        captured += B[opp_outer][c_f]
```

```
B[opp_outer][c_f] = 0

player_captured_total[player] += captured
```

The requirement that the landing hole must have been empty before receiving the final stone ensures that only productive terminal landings produce captures. This is a crucial game-mechanical distinction: a turn that ends by placing a stone into an already occupied inner row hole does not yield a capture, incentivising players to track which inner row holes are currently empty on both sides of the board.

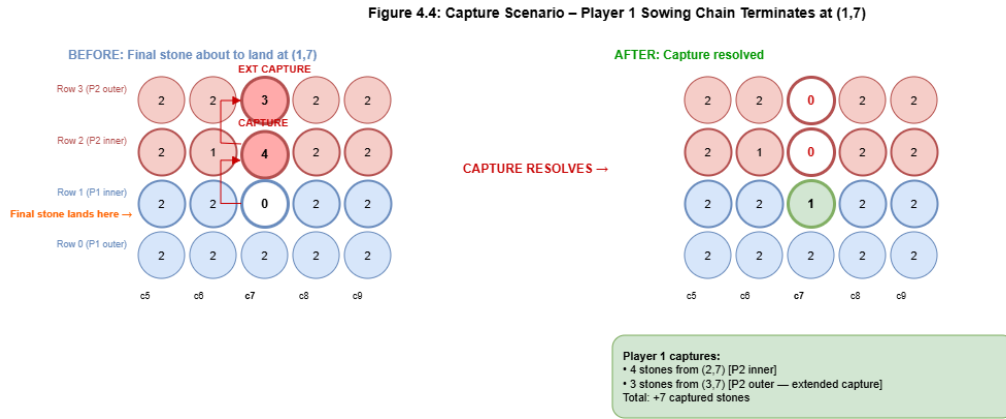


Figure 4.3: Capture scenario diagram showing Player 1's sowing chain terminating at inner row hole (1, 7), which was empty. Arrows indicate the capture of stones from Player 2's inner row hole (2, 7) and outer row hole (3, 7).

4.2.9 Game Termination and Win Condition

The game terminates under either of two conditions:

- Depletion: a player has no stones remaining in any of their holes, i.e., $\sum B[r][c] = 0$ for all r in $\text{player_rows}(p)$.
- No valid moves: a player's $\text{LEGAL_MOVES}(B, p)$ is empty, which may occur even if stones remain if all of the player's holes somehow became empty through captures (this is the same as the depletion condition in practice).

Game termination is checked at the beginning of each player's turn, after the previous player's move has been fully resolved including relay sowing and capture.

The winner is determined as follows:

1. If a player's stones are depleted, the opposing player wins regardless of captured stone counts.

2. If neither player's stones are depleted but the game has been declared over through an agreed draw or timeout (not standard in the formalised rules but included for implementation robustness), the player with the greater captured stone total wins.
3. In the event of an equal captured total under condition 2, the game is declared a draw.

```
function check_termination(B, p):
    stones = sum(B[r][c] for r in player_rows(p) for c in 0..11)
    if stones == 0:
        return GAME_OVER, winner = opponent(p)
    if LEGAL_MOVES(B, p) is empty:
        return GAME_OVER, winner = opponent(p)
    return GAME_CONTINUES
```

4.2.10 Summary of Formal Specification

Table 4.1 summarises the full formal specification of Nsolo as implemented in this system. It serves as the canonical reference for the game engine implementation in Section 4.5.

Rule Element	Formal Definition	Implementation Notes
Board	$B \in \mathbb{Z}_{\geq 0}^{(4 \times 12)}$	Integer array; no negative values permitted.
Player 1 rows	$\{0 \text{ (outer)}, 1 \text{ (inner)}\}$	Row 1 is adjacent to Player 2.
Player 2 rows	$\{2 \text{ (inner)}, 3 \text{ (outer)}\}$	Row 2 is adjacent to Player 1.
Initial state	$B[r][c]=2 \quad \forall (r,c);$ $B[1][11]=0; B[2][0]=0$	Rightmost inner hole starts empty.
Sowing direction	<i>Counterclockwise via</i> <i>PATH (48-element loop)</i>	PATH defined once at init; modulo 48 wrap.
Legal move	$B[r][c] \geq 1 \text{ AND } r \in$ $player_rows(p)$	Checked at start of every turn.
Relay condition	$B[last_hole] > 1 \text{ after}$ <i>receiving final stone</i>	Loop until relay condition false.
Capture trigger	<i>Land in own inner row</i> <i>AND hole was empty</i>	$B[inner][c] = 1$ after sow confirms empty-before.

Capture scope	<i>Opponent inner row col c; outer row col c if >0</i>	Extended capture is standard in this model.
Turn end	<i>Last stone lands in hole empty before placement</i>	B[last_hole] = 1 after sow.
Termination	<i>Player's stones = 0 OR legal moves = $\emptyset$</i>	Checked at start of each player's turn.
Winner	<i>Player with stones remaining; else by stone count</i>	Opponent wins if active player has no moves.

Table 4.1: Formal specification summary for the Nsolo game implementation.

4.3 Analysis of the System

System analysis at the design stage re-examines the requirements and state space characterisation from Chapter 3 to identify the key design challenges that the architectural and detailed design must address. The focus of this section is on what the system must do before decisions are made about how it will be structured.

The Nsolo game system must perform four primary functions: represent and maintain the game state across the full 4×12 board with accurate sowing, relay, and capture semantics (FR1-FR8); provide an AI opponent that selects moves of configurable strategic strength within the five-second time budget on Android hardware (FR9, NFR1); present the game to the human player through an interactive graphical interface with real-time board updates, score display, and embedded rule assistance (NFR2); and manage the game lifecycle including initialisation, turn sequencing, termination detection, and winner declaration (FR8, FR10).

Three design tensions emerge from this analysis. First, there is a tension between the correctness demands of the game engine and the performance demands of the AI agent. The engine must enforce complex rules exactly, but the AI agent calls the engine for every node in its search tree, meaning any inefficiency in rule processing is multiplied across potentially thousands of evaluations per turn. The design must therefore prioritise both correctness and computational efficiency in the game engine, with particular attention to the representation of board state and the implementation of sowing traversal.

Second, there is a tension between the independence of the AI algorithm from the presentation layer and the need for the Unity interface to reflect game state changes smoothly. The design separates these concerns explicitly: the game engine and AI agent are implemented as a pure C# library with no Unity dependencies, and the Unity layer communicates with the engine through a defined controller interface. This separation enables independent testing of the engine before integration and allows future extension to different front-end platforms without modifying the core logic.

Third, the requirement for three difficulty levels (FR9) requires that the AI search depth be tunable without restructuring the algorithm. The design isolates the difficulty parameter as a configuration value passed to the AI agent at initialisation, allowing the search depth, random move probability, and time budget to be varied independently for each level without affecting the minimax implementation.

4.4 Context Model of the System

The context model defines the boundary of the Nsolo game system and identifies the external entities that interact with it. The system is treated as a single unit at this level; internal decomposition is addressed in the architectural and detailed design sections that follow.

The primary external entity is the Human Player, who interacts with the system through touch input on an Android device. The player selects holes, triggers moves, and receives visual and textual feedback through the Unity user interface. The AI Agent, while implemented as a component of the system, is modelled at the context level as a distinct entity that receives game state from the Game Controller and returns a selected move. The Android Operating System provides the execution environment, memory management, and input event dispatch. The Unity Engine mediates between the application logic and the hardware rendering pipeline.

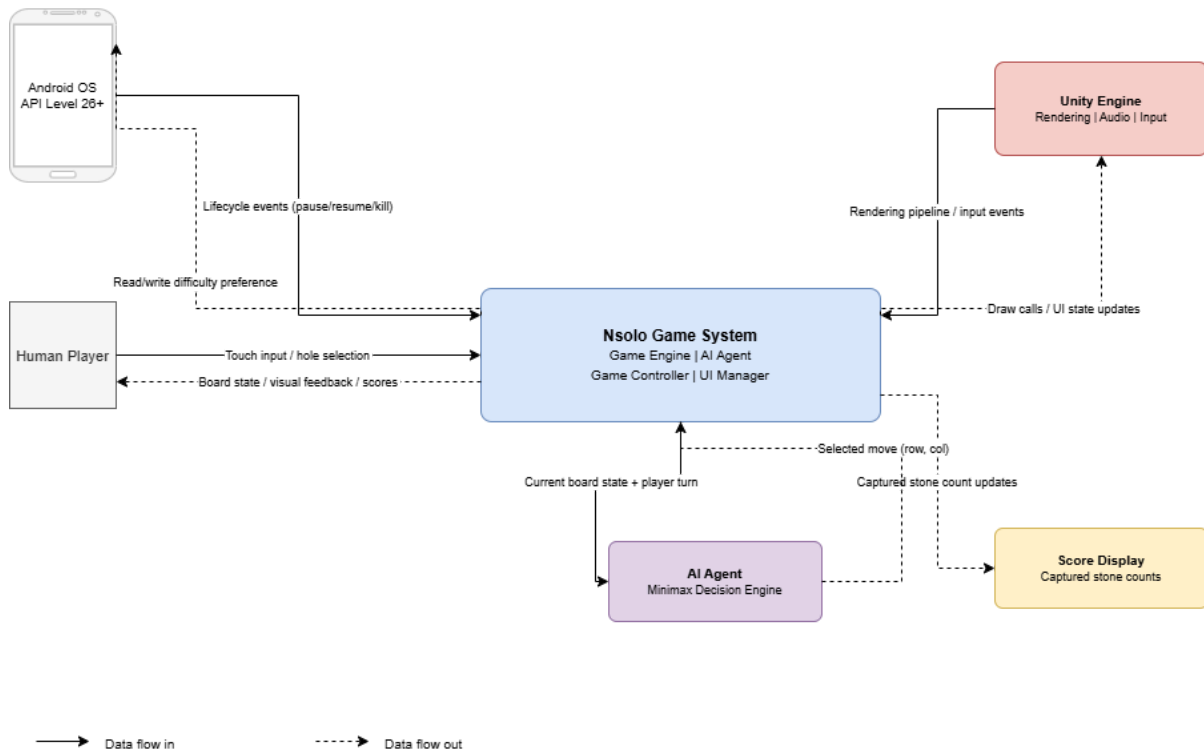


Figure 4.4: Context Diagram for the Nsolo Game System showing external entities (Human Player, Android OS, Unity Engine) and the primary data flows into and out of the system boundary.

The data flows across the system boundary are as follows. The Human Player sends touch input events representing hole selections, which are received by the Input Handler component within the system. The system sends rendered board state updates, score information, and feedback messages to the Human Player through the display. The Android OS provides lifecycle events such as application pause, resume, and termination, which the system must handle gracefully to preserve game state. The Unity Engine provides the rendering, audio, and input subsystems upon which the presentation layer depends.

4.5 Design Methods

4.5.1 Architectural Design

The architectural design describes the high-level decomposition of the system into subsystems and the communication patterns between them. The selection of an appropriate architectural style is the primary decision at this level.

Four architectural styles were considered: client-server architecture, layered architecture, repository architecture, and the model-view-controller (MVC) pattern. Client-server architecture was rejected on the basis that the Nsolo application is a single-device application

with no network communication requirement in its current scope. Repository architecture was considered but judged less appropriate than a layered approach because the system's primary complexity lies in its processing logic rather than in shared data management across loosely coupled subsystems.

The layered architecture was selected as the primary organisational style. This style partitions the system into layers with well-defined interfaces where each layer depends only on the layer immediately below it. The selected architecture uses four principal layers: the User Interface Layer, the Game Logic Layer, the AI Module, and the Android Platform Layer. This matches the separation of concerns identified in the design tension analysis of Section 4.2 and is consistent with Unity-based game architectures documented in Tsalapatas et al. (2022).

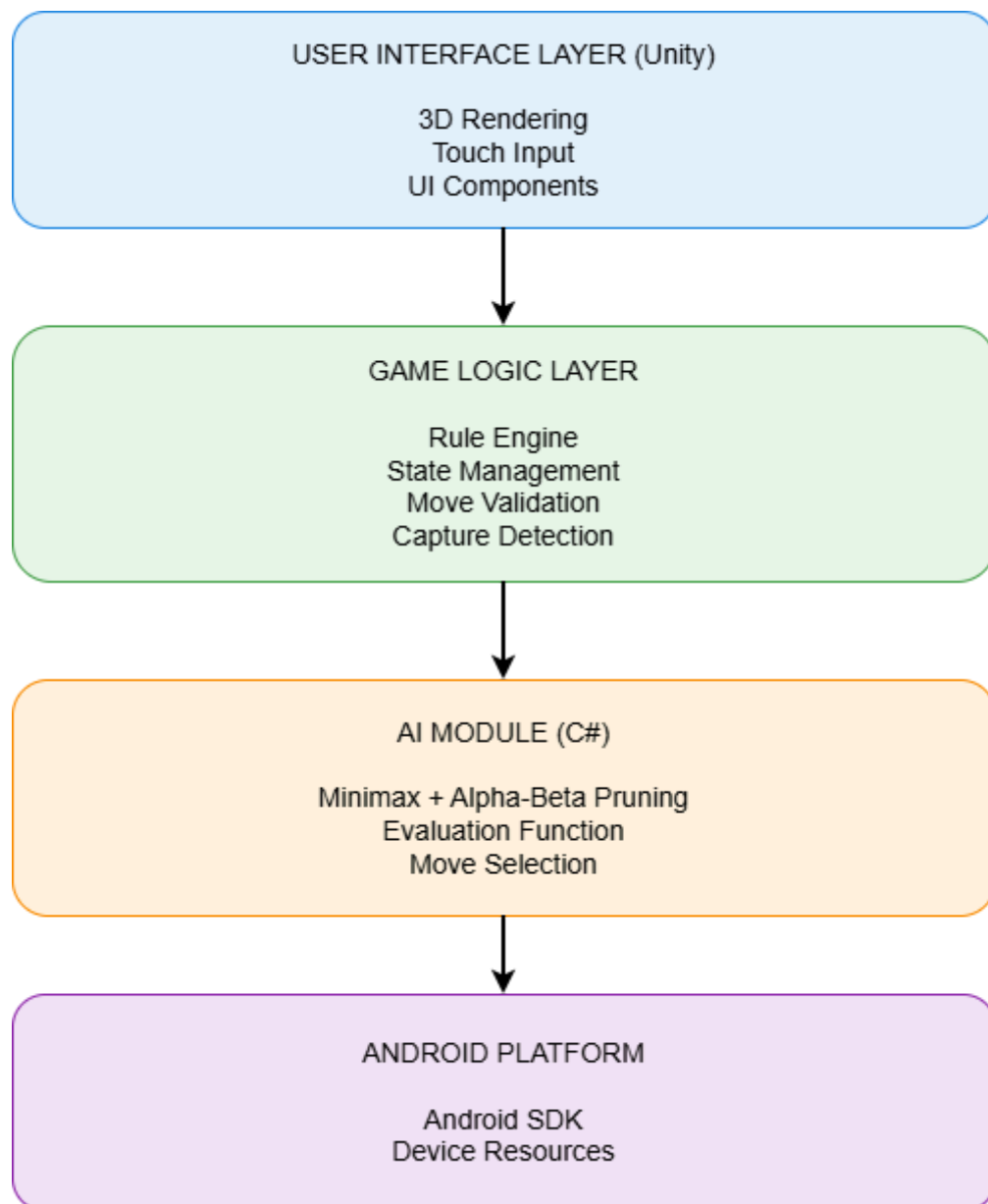


Figure 4.5: Architectural Diagram of the Layered Architecture of the Nsolo Game System showing the Presentation Layer (Unity UI), Application Logic Layer (Game Controller, Game Engine, AI Agent), and the Android Platform.

The Presentation Layer comprises all Unity-specific components, including the board renderer, input handler, animation system, and user interface elements. This layer is responsible solely for visualisation and user interaction and contains no game rule logic. All three-dimensional assets used in this layer, including the board and stone models, are created and optimised in Blender before being imported into Unity, ensuring efficient rendering on mobile hardware.

The Application Logic Layer contains the core computational components of the system: the GameController, GameEngine, and AI Agent. The GameEngine enforces all game rules, including sowing, relay, capture, and termination, while the AI Agent implements the minimax decision-making algorithm. This layer is implemented as a Unity-independent C# library, allowing it to be tested and verified independently of the presentation layer.

The Platform Layer consists of the Unity runtime environment and the Android operating system, which together provide rendering, input handling, memory management, and application lifecycle support. This layer abstracts the underlying hardware and enables consistent execution across Android devices.

4.5.2 Detailed Object-Oriented Design

The detailed design decomposes the Application Logic Layer into object classes with defined responsibilities and interfaces. The OOAD approach is used throughout, with UML class diagrams and sequence diagrams as the primary modelling artefacts.

The principal classes in the design are described below, followed by the class diagram and the key interaction sequences.

GameBoard is the central data class. It encapsulates the 48-integer array representing the board state, the two captured stone count integers, and the current player flag. It provides methods for cloning the board state (required for minimax tree exploration), querying hole contents, and computing the board's hash value for transposition table lookup. It does not enforce game rules; it is a pure data container.

GameEngine is the rules enforcement class. It receives a GameBoard and a Move and produces a new GameBoard reflecting the outcome of the move. Its primary methods are

GetLegalMoves(board, player), which returns all valid moves for the given player in the given state; ApplyMove(board, move), which executes sowing, relay resolution, and capture resolution in sequence and returns the resulting board state; and IsTerminal(board), which detects game-end conditions. GameEngine is stateless: it takes a board as input and returns a board as output, with no persistent state of its own. This design makes it straightforward to use GameEngine nodes in minimax tree exploration.

SowingPath is a utility class that pre-computes and stores the ordered list of (row, column) hole indices for the counterclockwise sowing traversal path starting from each of the 48 possible source holes. Pre-computing these paths at initialisation eliminates the need for boundary-condition logic during game tree search, where sowing path calculation would otherwise be repeated at every node. SowingPath is instantiated once and shared between GameEngine instances via dependency injection.

AIAgent implements the minimax algorithm with alpha-beta pruning. It is configured at construction with a difficulty level that determines the search depth, the time budget, and whether random move injection is active. Its public interface exposes a single method: SelectMove(board, player), which returns the best move found within the configured constraints. Internally, AIAgent uses a TranspositionTable to cache evaluated positions and applies move ordering to maximise pruning effectiveness. The evaluation function is encapsulated in a separate EvaluationFunction class to allow independent testing and potential future replacement.

EvaluationFunction computes the heuristic value of a non-terminal board state from the perspective of a specified player. It implements the weighted linear combination of five features defined in Section 3.6.2: stone count differential, captured stone differential, inner row mobility, capture threat score, and relay potential. The weights are stored as configurable constants, allowing adjustment without modifying the algorithm.

GameController orchestrates the game lifecycle. It initialises the GameBoard and SowingPath, coordinates turns between the human player and the AIAgent, invokes GameEngine to apply moves and detect termination, and communicates state changes to the Presentation Layer via a defined callback interface. It handles the asynchronous execution of the AI move computation to prevent blocking the Unity main thread during search.

UIManager is the primary Presentation Layer class. It receives state update events from GameController and updates the visual representation of the board, score counters, and

feedback messages. It also captures touch input, validates that a hole was selected, and forwards the selection to GameController for rule checking.

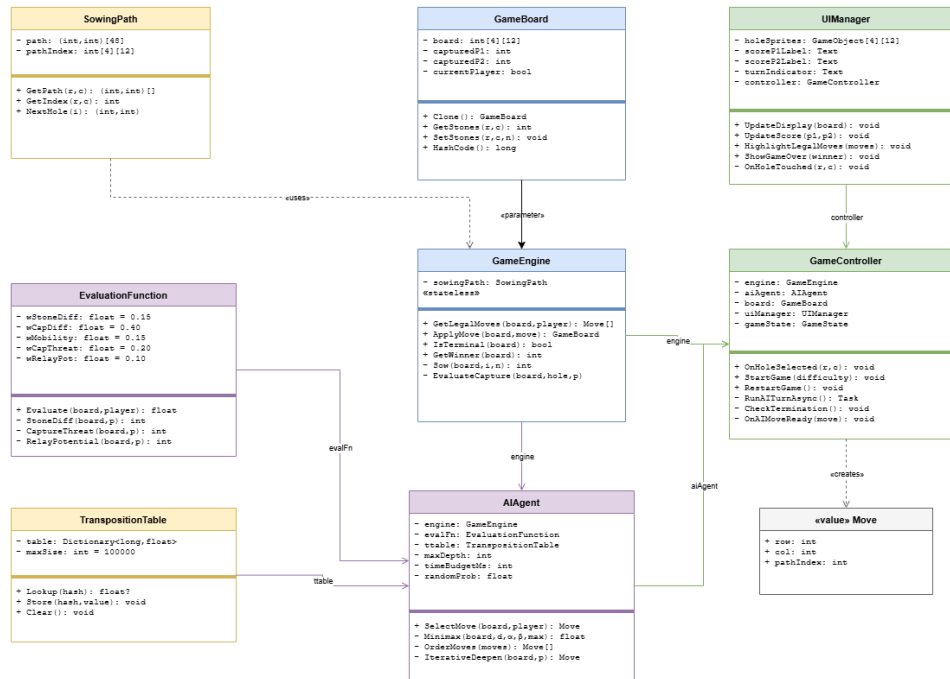


Figure 4.6: UML Class Diagram showing GameBoard, GameEngine, SowingPath, AI Agent, EvaluationFunction, GameController, and UIManager with their attributes, methods, and associations.

4.5.3 Key Interaction Sequences

Two interaction sequences are documented here: the human player turn and the AI player turn, since these represent the primary runtime behaviours of the system.

For a human player turn, the UIManager receives a touch event and identifies the selected hole. It calls GameController.OnHoleSelected(row, column). GameController calls GameEngine.GetLegalMoves(currentBoard, humanPlayer) and checks whether the selected hole corresponds to a legal move. If the move is legal, GameController calls GameEngine.ApplyMove(currentBoard, selectedMove), which returns an updated GameBoard. GameController updates its internal state with the new board and calls UIManager.UpdateDisplay(newBoard, capturedStones) to reflect the change. GameController then checks GameEngine.IsTerminal(newBoard); if the game has not ended, control transfers to the AI player turn.

For the AI player turn, GameController calls AI Agent.SelectMove(currentBoard, aiPlayer) on a background thread to avoid blocking the UI. AI Agent executes iterative deepening minimax with alpha-beta pruning, querying EvaluationFunction and consulting TranspositionTable at

each node. When the search completes or the time budget is exhausted, SelectMove returns the best move found. GameController receives the result on the main thread, calls `GameEngine.ApplyMove(currentBoard, aiMove)`, and updates UIManager. Termination is checked again before returning control to the human player.

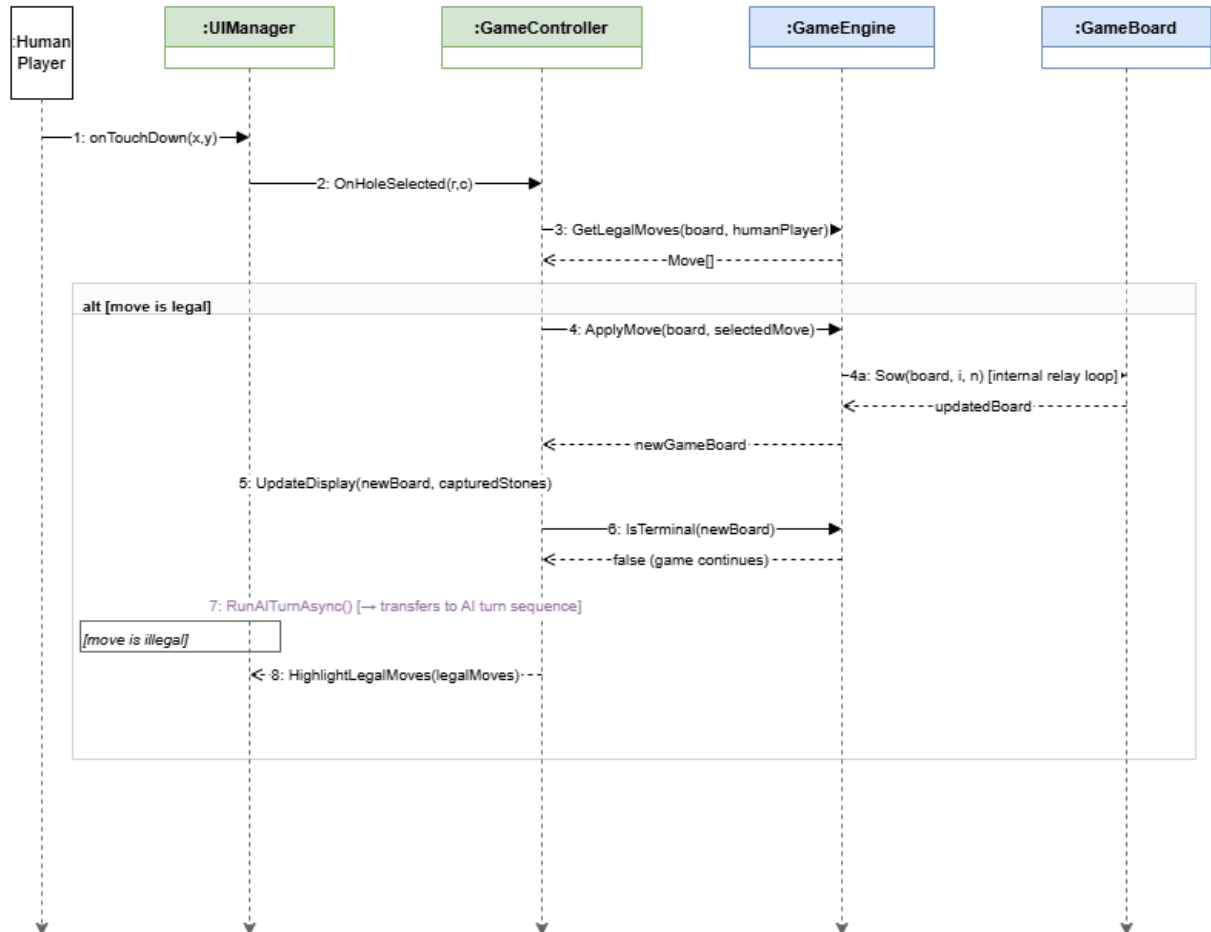


Figure 4.7: UML Sequence Diagram of Human Player Turn showing interactions between UIManager, GameController, GameEngine, and GameBoard.

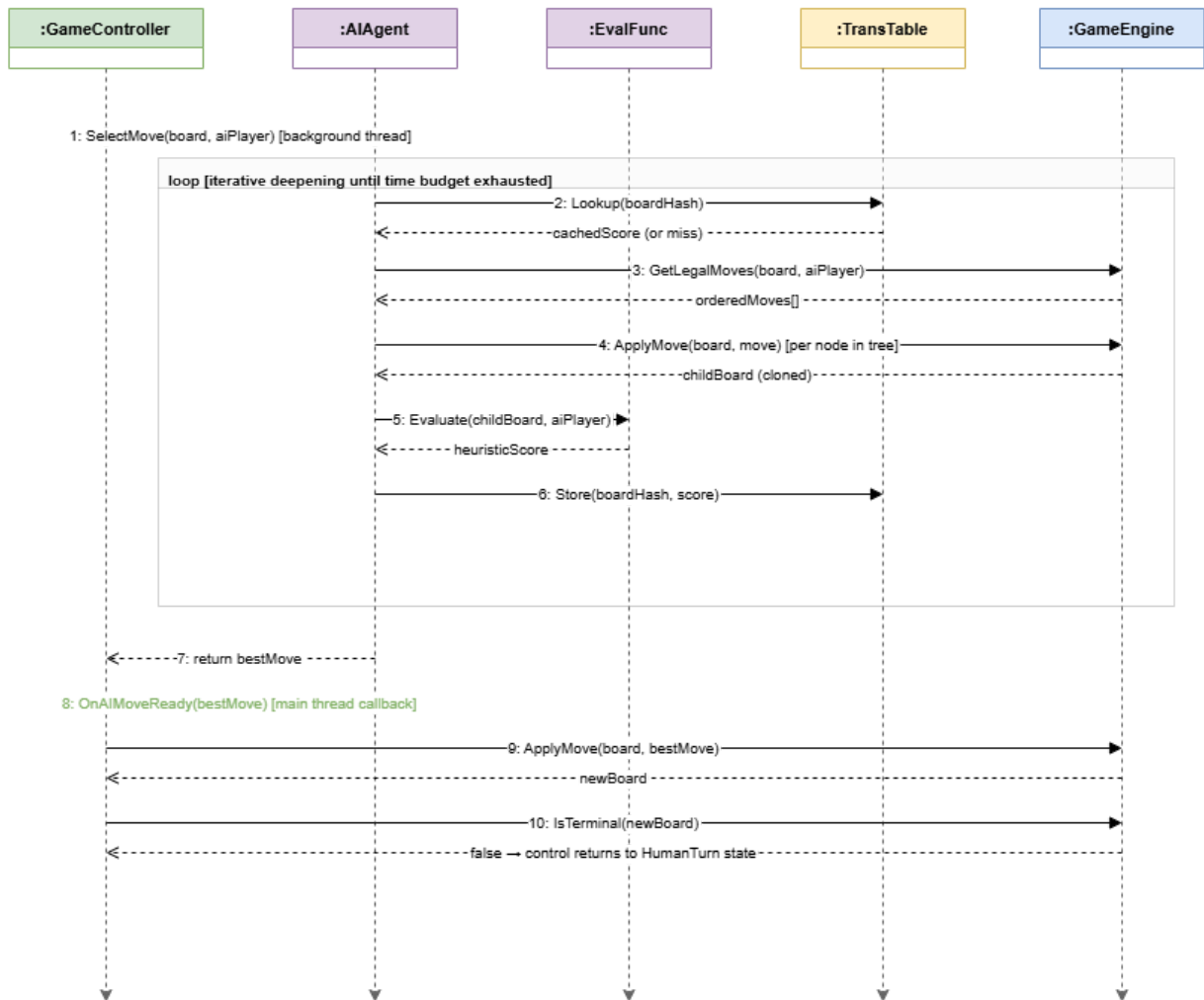


Figure 4.8: UML Sequence Diagram of AI Player Turn showing interactions between GameController, AI Agent, EvaluationFunction, TranspositionTable, and GameEngine.

4.5.4 State Machine Design

The game lifecycle is modelled as a finite state machine to clarify the valid transitions between game states and ensure that the GameController handles all conditions correctly.

The state machine defines six states: Initialising, in which the board and path data are set up; HumanTurn, in which the system awaits a hole selection from the player; ValidatingMove, in which GameEngine checks the legality of the selected hole; AiThinking, in which the AI agent is computing its move on a background thread; ApplyingMove, in which GameEngine processes the sowing, relay, and capture logic; and GameOver, in which the result is displayed, and the player may choose to restart. The transitions between these states are triggered by user input events, AI move completion events, and the return values of GameEngine methods.

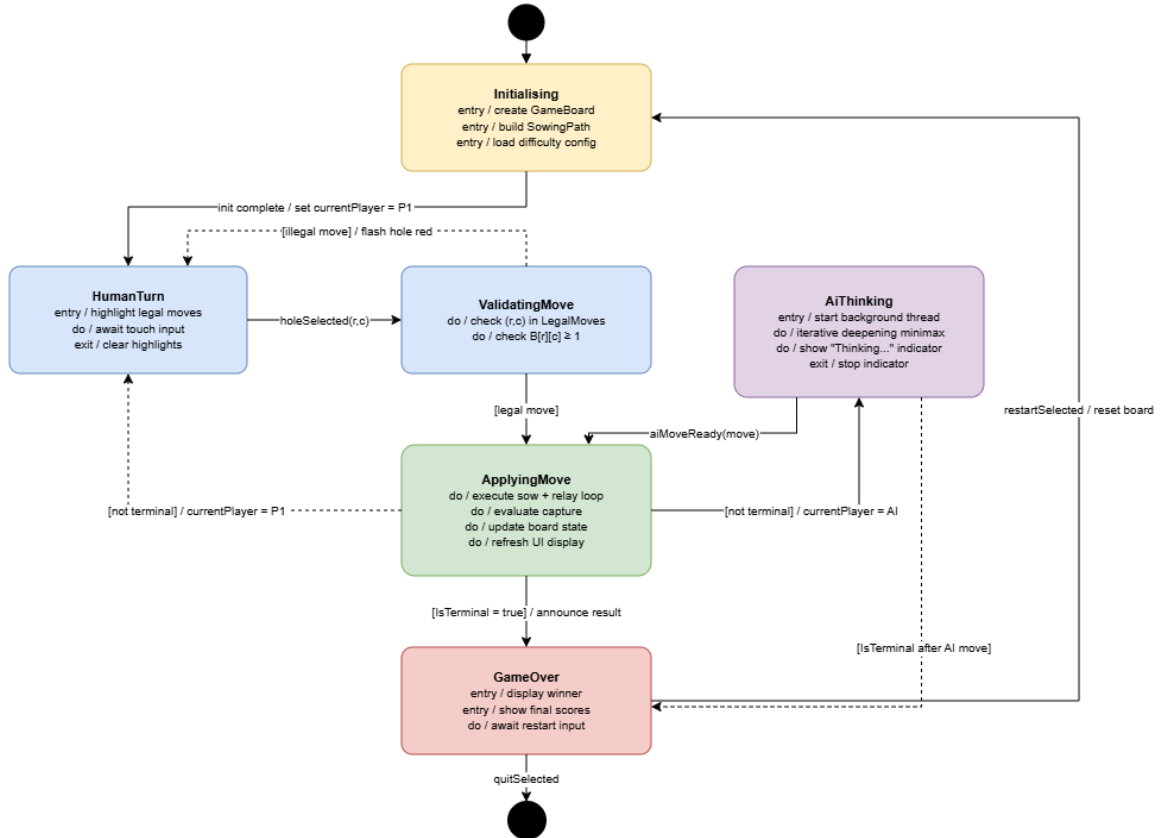


Figure 4.9: State Machine Diagram for the Nsolo Game Controller showing the six game states and the events that trigger transitions between them.

4.6 Physical Design

4.6.1 User Interface Design

The user interface presents the 4×12 Nsolo board as a grid of circular hole representations arranged in four rows. Player 1 controls the bottom two rows and Player 2 (AI) controls the top two rows. Each hole displays its current stone count as a numeric label centred within it. Holes belonging to the current player that contain at least one stone are highlighted to indicate valid move options. When a hole is selected, an animation illustrates the sowing sequence hole by hole to help the player follow the game mechanics.

The score area is displayed at the top and bottom of the board, showing each player's captured stone count in real time. A turn indicator shows whose turn is active. A menu accessible from a persistent icon provides access to rules explanation, difficulty selection, and game restart. The rule hints are presented as pop-up overlays triggered by the player, explaining sowing direction, relay behaviour, and capture conditions with annotated board diagrams.

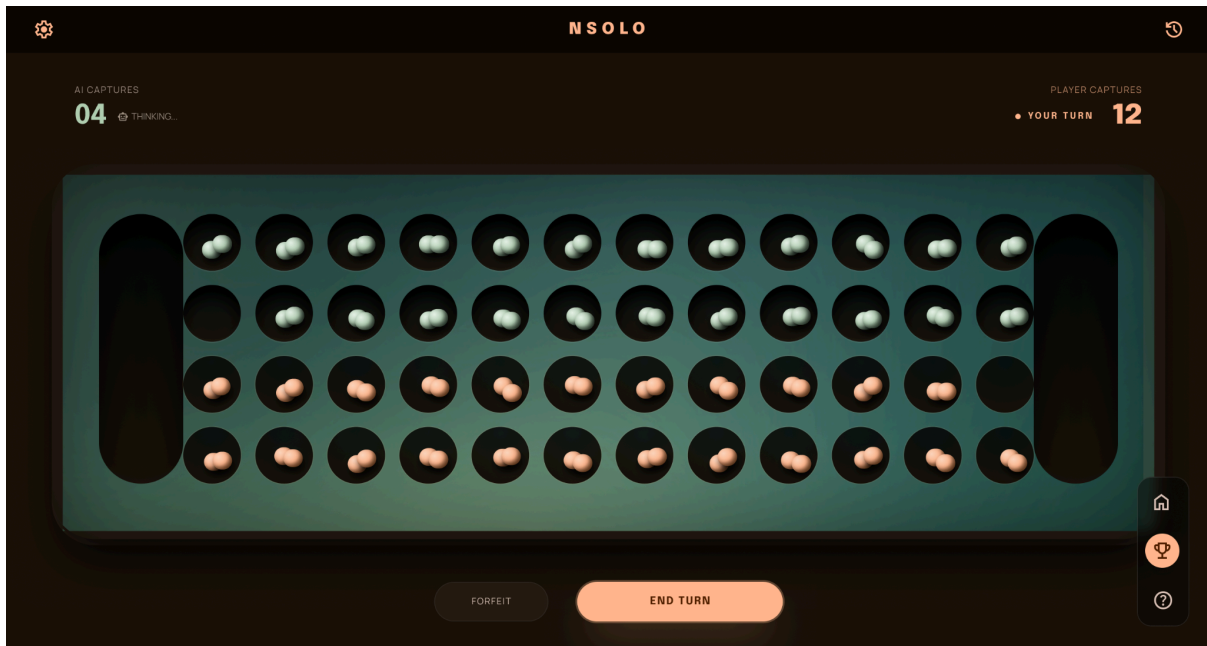


Figure 4.10: User Interface Wireframe showing the 4×12 board layout, score counters, turn indicator, and menu icon placement on a standard Android portrait screen.

4.6.2 Data Model

The persistent data model is minimal. No server-side persistence is required; all game state is held in memory during a session. The only persisted data is the user's difficulty preference, stored in Android SharedPreferences as a single integer value (0 = Easy, 1 = Medium, 2 = Hard). This value is read at application startup and used to configure the AI Agent.

The in-memory data model centres on the GameBoard class described in Section 4.4.2. The board state is represented as a flat integer array of length 48 (4 rows by 12 columns, indexed as $\text{row} * 12 + \text{column}$). Two integers store the captured stone counts for each player. A boolean records the current player. The SowingPath object holds a two-dimensional array of pre-computed traversal sequences, one per source hole.

4.6.3 Data Flow Through the System

Data flows through the system in a well-defined pattern that reflects the layered architecture. User input originates at the UIManager, which translates touch coordinates into hole indices. These indices are passed as Move objects to the GameController. GameController passes the current GameBoard and Move to GameEngine, which computes the updated board state through the sowing, relay, and capture logic. The updated GameBoard is returned to GameController, which distributes it both to UIManager for display and to AI Agent as the starting state for the subsequent AI turn. AI Agent's search internally clones the GameBoard

repeatedly using `GameBoard.Clone()` to explore the game tree without modifying the authoritative game state held by `GameController`.

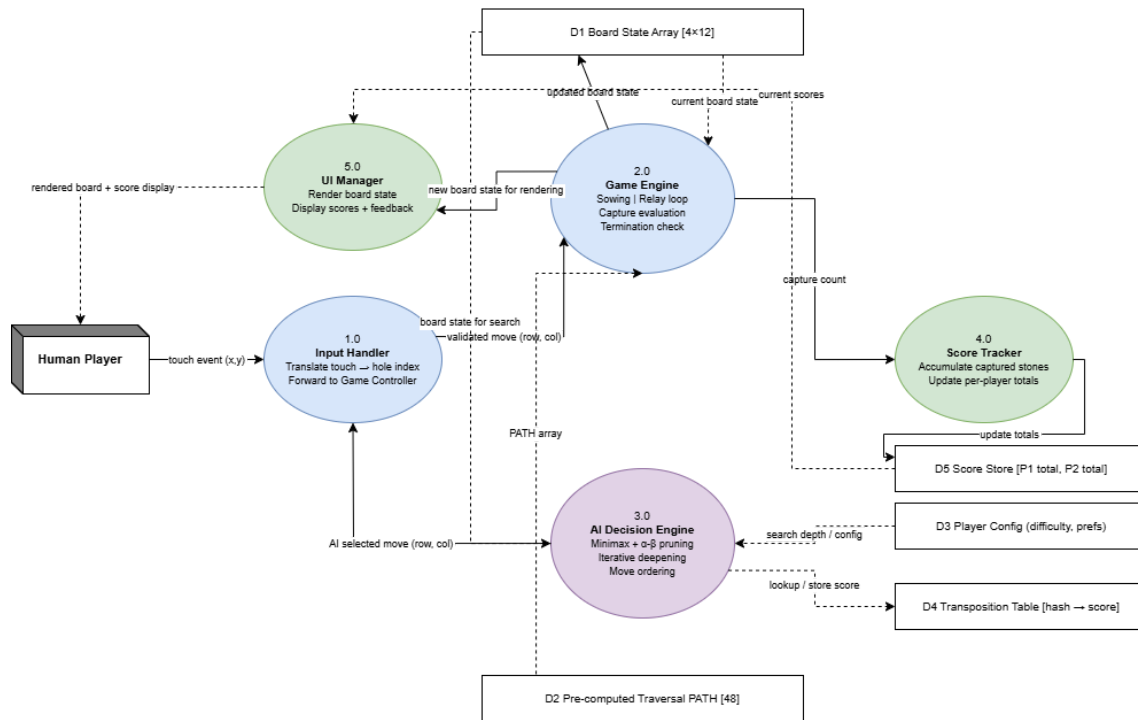


Figure 4.11: Data Flow Diagram (Level 1) for the Nsolo Game System showing the Input Handler, Game Engine, AI Decision Engine, and Score Tracker processes with their data stores and flows.

4.7 Conclusion

This chapter has presented the complete system design for the Nsolo intelligent agent application. The architectural design adopts a three-layer structure separating the Unity presentation layer from the game engine and AI logic, ensuring that correctness-critical components can be developed and tested independently of the graphical environment. The object-oriented detailed design defines seven principal classes with clearly specified responsibilities and interfaces, minimising coupling between the game rule enforcement, AI decision-making, and user interface components.

The context model, state machine, and UML sequence diagrams provide a comprehensive specification of the system's runtime behaviour that directly guides the implementation phase. The physical design defines a minimal data model appropriate to the single-device scope of the application and a user interface layout that prioritises clarity of the game state for both novice and experienced players.

The design satisfies all functional requirements (FR1–FR10) and the non-functional requirements (NFR1–NFR4) established in Chapter 3. In particular, the separation of the GameEngine as a Unity-independent library addresses NFR3 (correctness testing before integration), the configurable AI Agent depth addresses NFR1 (AI response time), the layered architecture enables NFR4 (portability), and the embedded rule-hint system addresses NFR2 (usability). The subsequent Chapter 5 documents how this design is translated into the implemented system.

REFERENCES

- Tsalapatas, A., Tsalis, P., Gkournelos, C., Stamoulias, A., Katsaounidou, A., Giannakelis, P., Vrysis, L., Floros, A., Dimoulas, C., & Koziri, M. (2022). Improving mobile game performance with basic optimization techniques in Unity. *Technologies*, 10(2), 48. <https://doi.org/10.3390/technologies10020048>
- Russell, S., & Norvig, P. (2010). *Artificial intelligence: A modern approach* (3rd ed.). Prentice Hall.
- Hunter, T. J. (2021). The exploration and analysis of Mancala from an AI perspective [Honours thesis, Andrews University]. Digital Commons @ Andrews University. <https://digitalcommons.andrews.edu/honors/257/>
- Wooldridge, M. (2009). *An introduction to multiagent systems* (2nd ed.). John Wiley & Sons.
- Politecnico di Milano. (2016). Design of artificial intelligence for Mancala games [Master's thesis]. Politecnico di Milano.